

Приложение 6. Описание проекта к КТ1

(программный проект)

1. Основные планы и этапы проекта

1.1. Краткое описание проекта

Название проекта:

- Natural Language Video Search System
- Система поиска информации в видеопотоках по естественному языку

Цель проекта:

Автоматический поиск людей, объектов и событий в видеозаписях по текстовому описанию или фотоизображению лица без необходимости ручного просмотра видео.

Описание проекта:

Разрабатываемая система предназначена для семантического (диалогового) поиска по видеоданным. Пользователь формирует текстовый запрос на естественном языке (например, «Покажи Иванова на Платонова 20б за январь» или «Синие BMW вчера») или прикладывает фотоизображение лица, после чего система автоматически находит релевантные фрагменты видеозаписей и предоставляет их с превью и точными таймкодами.

Система основывается на использовании методов компьютерного зрения и обработки естественного языка. Запросы обрабатываются большой языковой моделью (Qwen3-8B через vLLM) с режимом структурированного вывода, в результате которого извлекаются именованные сущности и фильтры запроса. Именованные сущности (фамилии, адреса, номерные знаки) преобразуются в эмбединги через модель **multilingual-e5-base** и ищутся в векторной базе данных Qdrant. Для поиска по фотоизображению внешняя VMS формирует векторный дескриптор лица, по которому выполняется поиск зарегистрированных лиц. Видеоматериалы и аналитические события поступают из внешней системы видеонаблюдения (VMS) через её программный интерфейс.

Архитектура проекта реализована как стейтфул-агент на LangGraph с поддержкой механизма интерактивного уточнения неоднозначных сущностей (interrupt / resume): при обнаружении нескольких подходящих кандидатов граф приостанавливается, и пользователь выбирает нужный вариант, после чего обработка продолжается.

Параллельно проведена исследовательская работа (research) по выбору оптимальной модели для будущей самостоятельной индексации видео, включая:

- агрегацию эмбедингов видеокадров и оконные стратегии индексации;
- мультимодальные модели (CLIP-подобные: CLIP ViT-L/14, X-CLIP);
- модели прямой локализации видеомоментов (CG-DETR в режимах прямой локализации и cross-encoder saliency);
- LLM-эмбединги для видео (Qwen3-VL-Embedding-2B/8B) с перанкером Qwen3-VL-Reranker-2B.

1.2. Планы и этапы выполнения проекта

Этап	Содержание работ	Результат	Сроки
Анализ и ресерч	Исследование и сравнение архитектур семантического поиска по видеоданным, выбор базового подхода, бенчмарк retrieval-моделей на QVHighlights	Описанная архитектура системы, выбранный baseline-метод, сравнительная таблица моделей по метрикам mean_iou , Recall@K , MR@IoU , MRR , mAP	01.12.2025
			– 28.02.2026
Проектирование	Проектирование серверной архитектуры, LangGraph-графа диалогового поиска и схем хранения данных	Архитектурная схема и логические схемы БД	01.03.2026
			– 05.03.2026
Инфраструктура хранения	Развёртывание хранилищ метаданных, эмбеддингов сущностей и пользовательских фотоизображений	Рабочая инфраструктура в Docker Compose	06.03.2026
			– 10.03.2026
Backend	Реализация серверного API: авторизация, чат-сессии, LangGraph-пайплайн диалогового поиска	HTTP API с автодокументацией	11.03.2026
			– 25.03.2026
ML-пайплайн	Парсинг запросов через LLM, поиск сущностей в Qdrant, поиск по фото через face descriptor VMS, формирование ответа на основе аналитических событий VMS	Работающий пайплайн диалогового поиска	11.03.2026
			– 25.03.2026
Поиск и ранжирование	Реализация семантического поиска и ранжирования + потоковый вывод результата (SSE)	Поиск с временем первого токена ≤ 5 с и полного ответа ≤ 60 с	26.03.2026
			– 31.03.2026
Frontend	Vue 3 SPA с поддержкой Server-Sent Events и интерактивного выбора кандидатов	Пользовательский интерфейс системы	01.04.2026
			– 10.04.2026
Тестирование	Тестирование компонентов и стабилизация системы	Набор unit- и integration-тестов и стабильная версия проекта	11.04.2026
			– 28.04.2026

2. Используемый технологический стек и его обоснование

2.1. Технологический стек

Компонент	Технологии
Backend	Python 3.11, FastAPI, LangGraph, LangChain Core
Frontend	Vue 3, TypeScript, Vite, Pinia, Vue Router, Axios, DOMPurify, markdown-it
LLM-инференс	vLLM (Qwen/Qwen3-8B), OpenRouter (резервный провайдер)
Эмбединги	Hugging Face Text Embeddings Inference (intfloat/multilingual-e5-base)
Хранение видео	Внешняя VMS (Video Management System), интегрируется через HTTP API
Объектное хранилище	MinIO (S3-совместимое) — для фотоизображений из чатов
Метаданные	PostgreSQL 16 + asyncpg + SQLAlchemy
Векторный поиск	Qdrant
Кеш и JWT-blacklist	Redis 7
Контейнеризация	Docker, Docker Compose
Шифрование секретов	SOPS + age
Видеообработка (research)	OpenCV, FFmpeg, PyTorch
Research-эксперименты	Jupyter, FAISS, ClearML
Тестирование	pytest (backend / ai), vitest + vue-tsc (frontend)

2.2. Обоснование выбора технологий

FastAPI используется для построения серверного API благодаря высокой производительности, поддержке асинхронности и автоматической генерации OpenAPI-документации (Swagger UI, ReDoc).

LangGraph выбран для реализации диалогового агента: фреймворк поддерживает контрольные точки графа и механизм **interrupt / resume**, необходимый для интерактивного уточнения неоднозначных сущностей.

vLLM с моделью Qwen3-8B обеспечивает высокопроизводительный инференс LLM с OpenAI-совместимым API и поддержкой структурированного вывода. **TEI с моделью multilingual-e5-base** даёт качественные эмбединги для русско-английских именованных сущностей с GPU-поддержкой.

S3-совместимое хранилище MinIO применяется для пользовательских фотоизображений в чатах. **PostgreSQL** — для реляционных данных (пользователи, чаты, сообщения) и LangGraph-чекпоинтов.

Векторная база Qdrant используется для быстрого семантического поиска по эмбедингам именованных сущностей (фамилии, адреса, номерные знаки) с поддержкой фильтров.

Redis обеспечивает кеширование и список отозванных JWT-токенов.

Vue 3 + TypeScript + Vite заменяет изначально планировавшийся прототипный фронтенд на полноценный SPA с поддержкой Server-Sent Events для потокового вывода ответа агента и интерактивных элементов выбора кандидатов.

Docker Compose обеспечивает воспроизводимое развёртывание всей системы на Linux-сервере;
SOPS совместно с age — безопасное хранение секретов в репозитории.

3. Требования к системе

3.1. Функциональные требования

- авторизация и регистрация пользователей с защищённым хранением учётных данных (JWT + bcrypt-хеши паролей);
- управление пользовательским аккаунтом, включая просмотр профиля и удаление аккаунта;
- ведение чат-сессий: создание, просмотр истории сообщений, удаление чата с каскадным удалением фотоизображений;
- интерпретация текстовых запросов на естественном языке через большую языковую модель;
- семантический поиск именованных сущностей (фамилий, адресов, номерных знаков) по эмбедингам в векторной базе данных;
- интерактивное уточнение неоднозначных сущностей через выбор пользователем одного или нескольких кандидатов из списка (interrupt / resume);
- поиск видеоаналитических событий по фотоизображению лица (получение дескриптора лица через VMS);
- интеграция с внешней системой видеонаблюдения (VMS) как источником видеоматериалов и аналитических событий;
- проксирование снимков из VMS пользователю с предварительной проверкой авторизации;
- формирование результатов поиска в виде превью видеоаналитических событий с временными метками и глубокими ссылок (deep-link) на видеосегменты во внешней VMS;
- веб-интерфейс с авторизацией пользователей, поддержкой потокового вывода (SSE) и интерактивных элементов выбора.

3.2. Нефункциональные требования

- время первого фрагмента ответа в потоковом режиме — не более 5 секунд при штатной нагрузке;
- время полного ответа на запрос — не более 60 секунд при штатной нагрузке;
- ограничение длительности обработки запроса в LangGraph-графе — 300 секунд (по истечении возвращается информативное сообщение об ошибке);
- минимальные ресурсы сервера: 16 ГБ оперативной памяти, центральный процессор и графический ускоритель NVIDIA с поддержкой CUDA;
- клиентская часть работает в современном браузере без установки дополнительного программного обеспечения;
- все компоненты системы развёртываются в Docker-контейнерах;
- система должна корректно обрабатывать некорректные пользовательские данные и сбои отдельных компонентов без аварийного завершения работы.

4. Критерии оценивания проекта

- использование реляционной БД (PostgreSQL) и векторной БД (Qdrant);
- наличие веб-интерфейса с авторизацией и регистрацией пользователей (4 экрана: `/auth`, `/chat`, `/profile`, `/status`);
- наличие автодокументации API (FastAPI Swagger UI / ReDoc);
- соответствие функциональным требованиям, указанным в п. 3.1;
- стабильность работы и воспроизводимость развёртывания (Docker Compose, SOPS-секреты, `make up` / `make test`);
- наличие автоматических тестов для ключевых компонентов (pytest для backend и AI-сервиса, vitest для frontend, integration-тесты);
- использование стейтфул-агента LangGraph и интеграция с внешней VMS API;
- наличие исследовательской части (notebooks `00–05`) с бенчмарком retrieval-моделей на датасете QVHighlights.